

A Practical Guide to Q-learning

TOPIC -- Q-LEARNING

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

-- 01 --

After Δt steps into the future the agent will decide some next step. The weight for this step is calculated as $\gamma \Delta t$, where γ (the discount factor) is a number between 0 and 1 ($0 \leq \gamma \leq 1$). Assuming $\gamma < 1$, it has the effect of valuing rewards received earlier higher than those received later (reflecting the value of a "good start"). γ may also be interpreted as the probability to succeed (or survive) at every step Δt . The algorithm, therefore, has a function that calculates the quality of a state–action combination:

-- 02 --

Discount factor The discount factor γ determines the importance of future rewards. A factor of 0 will make the agent "myopic" (or short-sighted) by only considering current rewards, i.e. r_t (in the update rule above), while a factor approaching 1 will make it strive for a long-term high reward. If the discount factor meets or exceeds 1, the action values may diverge. For $\gamma = 1$, without a terminal state, or if the agent never reaches one, all environment histories become infinitely long, and utilities with additive, undiscounted rewards generally become infinite. Even with a discount factor only slightly lower than 1, Q-function learning leads to propagation of errors and instabilities when the value function is approximated with an artificial neural network. In that case, starting with a lower discount factor and increasing it towards its final value accelerates learning.

-- 03 --

Reinforcement learning involves an agent, a set of states S , and a set A of actions per state. By performing an action $a \in A$, the agent transitions from state to state. Executing an action in a specific state provides the agent with a reward (a numerical score). The goal of the agent is to maximize its total reward. It does this by adding the maximum reward attainable from future

states to the reward for achieving its current state, effectively influencing the current action by the potential future reward. This potential reward is a weighted sum of expected values of the rewards of all future steps starting from the current state. As an example, consider the process of boarding a train, in which the reward is measured by the negative of the total time spent boarding (alternatively, the cost of boarding the train is equal to the boarding time). One strategy is to enter the train door as soon as they open, minimizing the initial wait time for yourself. If the train is crowded, however, then you will have a slow entry after the initial action of entering the door as people are fighting you to depart the train as you attempt to board. The total boarding time, or cost, is then:

-- 04 --

Learning rate The learning rate or step size determines to what extent newly acquired information overrides old information. A factor of 0 makes the agent learn nothing (exclusively exploiting prior knowledge), while a factor of 1 makes the agent consider only the most recent information (ignoring prior knowledge to explore possibilities). In fully deterministic environments, a learning rate of $\alpha_t = 1$ is optimal. When the problem is stochastic, the algorithm converges under some technical conditions on the learning rate that require it to decrease to zero. In practice, often a constant learning rate is used, such as $\alpha_t = 0.1$ for all t .

-- 05 --

Double Q-learning Because the future maximum approximated action value in Q-learning is evaluated using the same Q function as in current action selection policy, in noisy environments Q-learning can sometimes overestimate the action values, slowing the learning. A variant called Double Q-learning was proposed to correct this. Double Q-learning is an off-policy reinforcement learning algorithm, where a different policy is used for value evaluation than what is used to select the next action. In practice, two separate value functions Q_A and Q_B are trained in a mutually symmetric fashion using separate experiences. The double Q-learning update step is then as follows:

-- 06 --

$$Q^{\text{new}}(S_t, A_t) \leftarrow (1 - \alpha) \cdot Q(S_t, A_t) + \alpha \cdot (R_{t+1} + \gamma \cdot \max_a Q(S_{t+1}, a))$$
 ■ learning rate ■ current value + α ■ learning rate ■ reward + γ ■ discount factor ■ estimate of optimal future value ■ new value (temporal difference target)

$$\text{value}} + \underbrace{\alpha}_{\text{learning rate}} \cdot \underbrace{\bigg(\underbrace{R_{t+1}}_{\text{reward}} + \underbrace{\gamma}_{\text{discount factor}} \cdot \underbrace{\max_a Q(S_{t+1}, a)}_{\text{estimate of optimal future value}} \bigg)}_{\text{new value (temporal difference target)}}$$

-- 07 --

0 seconds wait time + 15 seconds fight time On the next day, by random chance (exploration), you decide to wait and let other people depart first. This initially results in a longer wait time. However, less time is spent fighting the departing passengers. Overall, this path has a higher reward than that of the previous day, since the total boarding time is now:

-- 08 --

History Q-learning was introduced by Chris Watkins in 1989. A convergence proof was presented by Watkins and Peter Dayan in 1992, building on Watkins' doctoral dissertation, Learning from Delayed Rewards. Eight years earlier in 1981, the same problem, under the name of "Delayed reinforcement learning," was solved by Bozinovski's Crossbar Adaptive Array (CAA). The memory matrix $W = w(a, s)$ was the same as the eight years later Q-table of Q-learning. The architecture introduced the term "state evaluation" in reinforcement learning. The crossbar learning algorithm, written in mathematical pseudocode in the paper, in each iteration performs the following computation: